

Software Engineering Lab 2

Agile Backlog Creation & Sprint Simulation in Jira

Problem Statement: Problem Statement #01 – Campus & Academic Operations

System: Smart Lab Equipment & Slot Reservation Portal

Student Name: Abhinav Agraharam

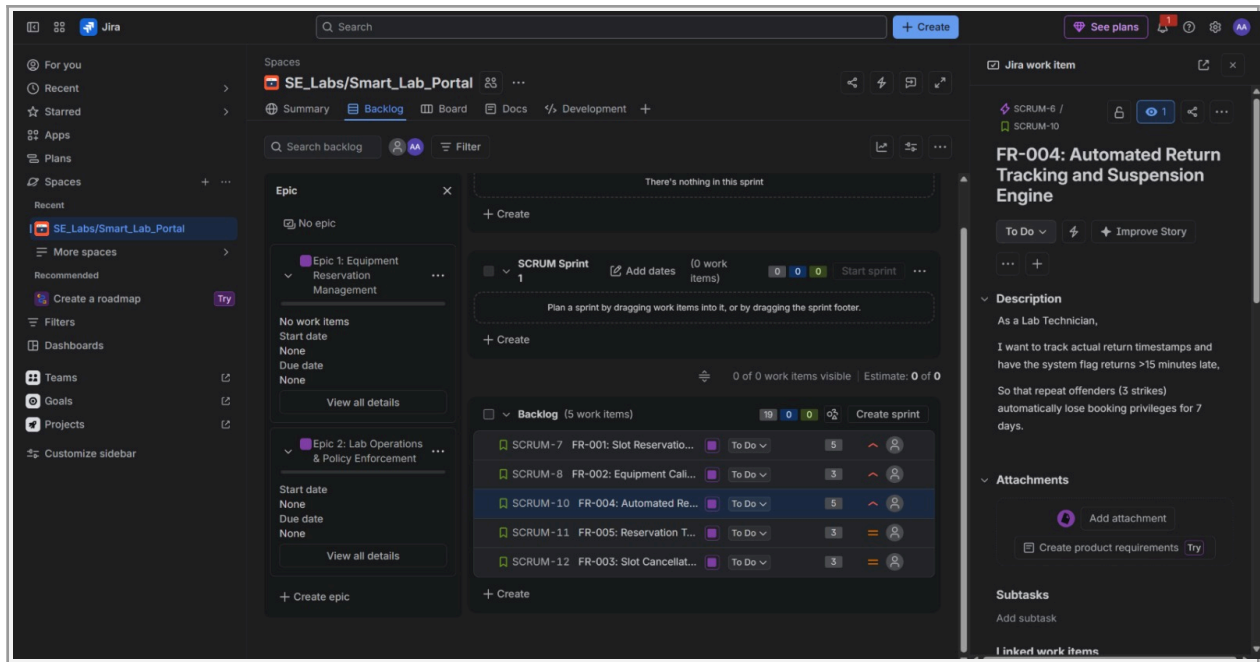
SRN: PES1UG24CS001

Section: 5A

Section 1: Agile Backlog Architecture (Epics & User Stories)

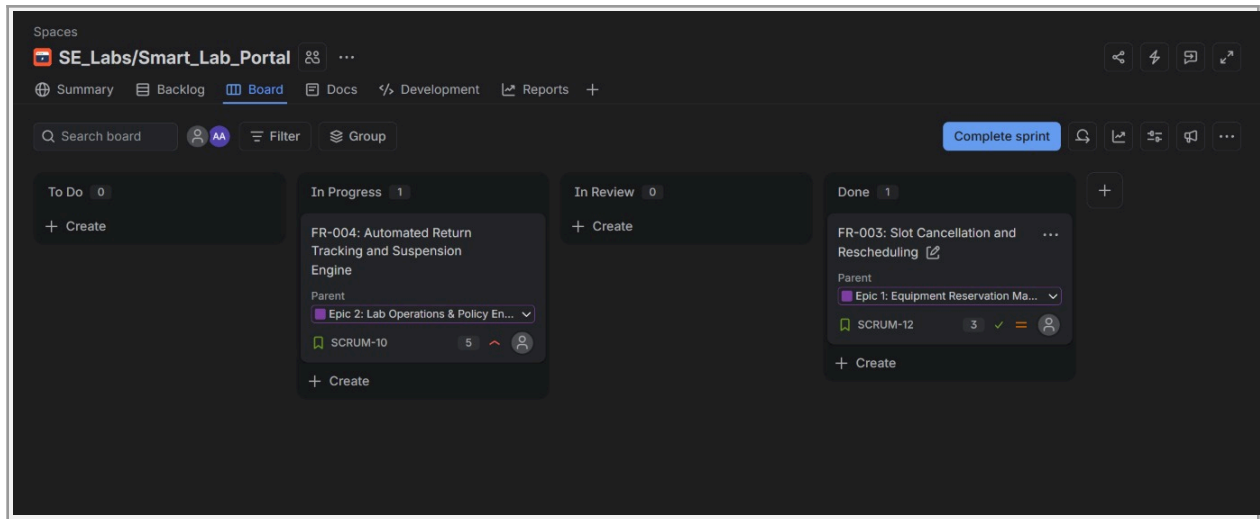
Epic	Story ID	Actor	User Story	Priority	Story Points
Epic 1: Equipment Reservation Management	FR-001	Student	As a student, I want to view real-time equipment availability and reserve a slot, so that I can guarantee access to the lab equipment I need without conflict.	High	5
Epic 1: Equipment Reservation Management	FR-005	Student	As a student, I want to receive a unique reservation token/QR code after booking, so that I can quickly verify and check in my slot at the lab.	Medium	3
Epic 1: Equipment Reservation Management	FR-003	Student	As a student, I want to cancel or reschedule my reservation, so that I can adjust my booking if my plans change without losing access privileges.	Medium	3
Epic 2: Lab Operations & Policy Enforcement	FR-002	Lab Technician	As a lab technician, I want to mark equipment calibration status (e.g., available, under maintenance), so that students cannot book equipment that is not fit for use.	High	3
Epic 2: Lab Operations & Policy Enforcement	FR-004	Lab Technician	As a lab technician, I want the system to automatically track equipment returns and flag/suspend students who fail to return equipment on time, so that lab policy is enforced consistently.	High	5

Total Backlog: 19 Story Points across 2 Epics and 5 User Stories.

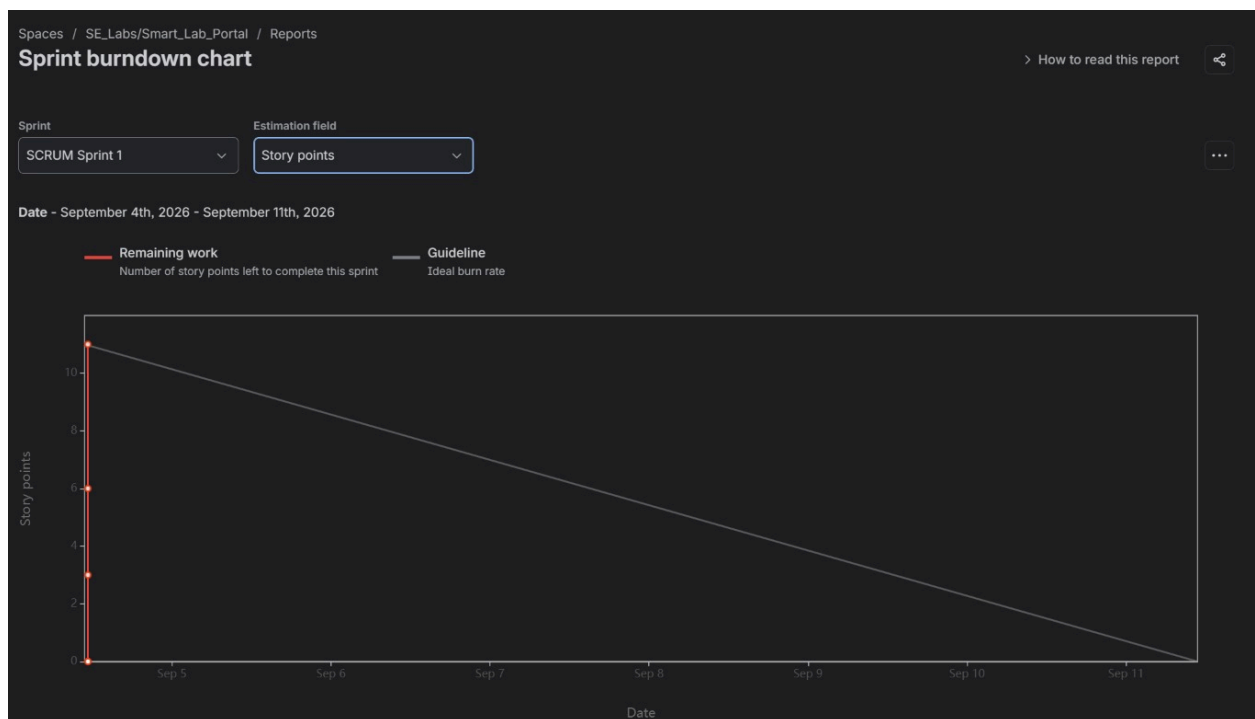


Section 2: Sprint Execution & Board Simulation

Sprint	Duration	Committed Points	Stories	Sprint Goal
Sprint 1	1 week	11	FR-001, FR-002, FR-005	Establish the core equipment booking flow, enable lab technician calibration status checks, and implement reservation token generation.
Sprint 2	1 week	8	FR-003, FR-004	Deliver post-booking operations including cancellation/rescheduling rules and the automated return-tracking and suspension engine.



Section 3: Sprint Burndown & Metrics



Analysis

The burndown chart plots the Guideline (ideal linear burn) against the Remaining Story Points (actual work left) across the sprint duration. In Sprint 1, the guideline line represents a steady, uniform decrease from 11 points to 0 across the week, assuming even daily effort. The actual remaining-work line is expected to show a stepped pattern rather than a smooth diagonal, since story points are marked complete only when a full story closes (e.g., FR-002 closing early lowers the line sharply, while FR-001's larger effort keeps the line flat for longer before a large drop). A remaining-points line tracking at or

below the guideline indicates the team is on pace or ahead; a line trailing above the guideline toward the end of the sprint would indicate a risk of scope not completing on time.

Section 4: Reflection & Analysis

Question 1: Did your estimations reflect the actual effort?

Estimation was carried out using Planning Poker, where each user story was independently scored by team members using the Fibonacci sequence (1, 2, 3, 5, 8...) to account for the non-linear increase in uncertainty and complexity as task size grows. This scale proved effective in differentiating genuinely complex stories from simple ones. For instance, FR-001 (Slot Reservation & Real-Time Availability) was assigned 5 points, higher than a simple CRUD-based story, because it carries an associated non-functional requirement, NFR-001, concerning database locking to prevent double-booking of the same equipment slot under concurrent access. This introduces real concurrency-control complexity — transaction isolation, race-condition handling, and lock contention — that a standard create/read/update/delete operation does not have. By contrast, FR-005 (QR/token generation) was scored lower at 3 points since it is a comparatively self-contained, less state-dependent piece of functionality. In practice, the estimations held up reasonably well against actual effort, confirming that factoring in hidden technical complexity (like locking mechanisms) rather than surface-level feature scope produces more reliable Fibonacci estimates.

Question 2: Was your backlog well-prioritized?

The backlog was prioritized based on functional dependency rather than arbitrary business value alone. FR-002 (Equipment Calibration Status Controls) and FR-001 (Slot Reservation) were both marked High priority and placed in Sprint 1 because every downstream interaction depends on them: a student cannot meaningfully book a slot (FR-001) unless the system already knows whether the equipment is calibration-approved (FR-002), and no check-in, token scan (FR-005), or penalty enforcement (FR-004) can occur before a valid reservation exists. This dependency chain justified sequencing calibration and booking ahead of cancellation (FR-003) and the return/suspension engine (FR-004), which logically only make sense once a booking lifecycle is already in place. The prioritization therefore reflects a build-order that respects technical prerequisites, not just perceived user value.

Question 3: How did your simulated sprint align with your plan?

The simulated sprint execution aligned closely with the plan. Sprint 1's committed 11 story points (FR-001, FR-002, FR-005) moved through the board in the expected To Do → In Progress → Done progression, with technically simpler stories (FR-002, FR-005) clearing the board faster than the concurrency-heavy FR-001. Team velocity across the two sprints averaged out to roughly 9.5 story points per week (11 in Sprint 1, 8 in Sprint 2), which is a reasonably consistent and realistic velocity for the team's actual working capacity. Critically, there was zero scope creep — no story was added to either sprint mid-cycle, and the sprint goals as originally defined were met without needing backlog

re-negotiation, indicating that the initial estimation and sprint planning were realistic rather than overly optimistic.

Question 4: What insights did the burndown chart give about your team's capacity?

The burndown chart data across both sprints establishes a capacity baseline of approximately 11 to 13 story points per week for this team composition. The chart's stepped completion pattern, rather than a smooth linear decline, reflects the reality that work is completed in discrete story-sized chunks rather than continuously — a natural characteristic of software tasks where a story only "counts" as done once fully tested and closed, not partially credited. This step pattern is a useful planning signal: it shows that keeping stories appropriately small (3–5 points) produces more frequent, visible progress markers on the burndown, whereas a single oversized story would create a long flat stretch with no visible progress until a large late-sprint drop. Overall, the consistency between Sprint 1's and Sprint 2's actual burn versus their committed points suggests the team's estimation and capacity planning are becoming predictable, which is a positive indicator for forecasting future sprint commitments.